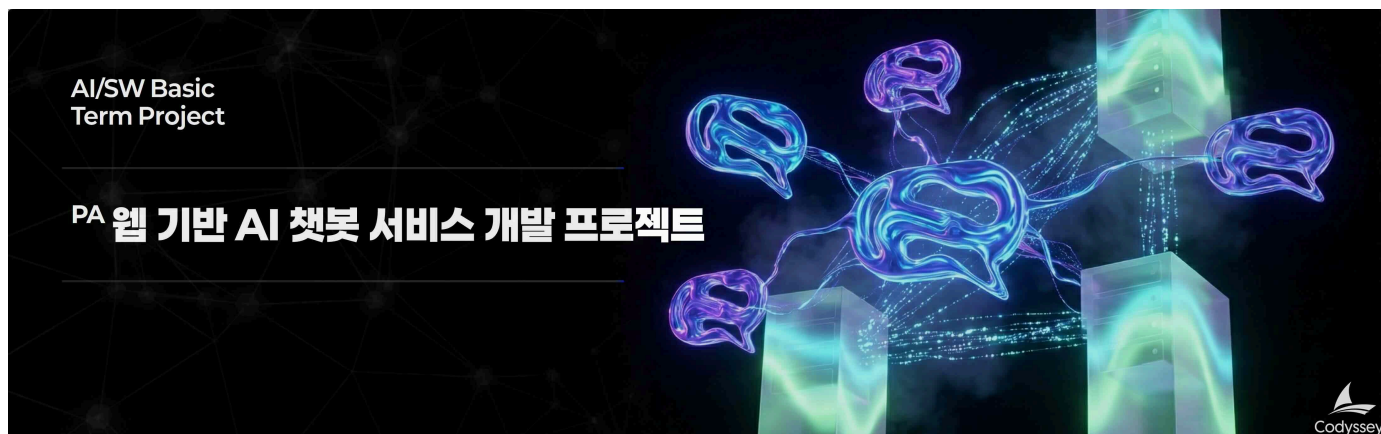


웹 기반 AI 챗봇 서비스 개발 프로젝트

문제기술

기술적 설명



1. 미션 소개

기술이 조각나 있으면 서비스가 안 됩니다. 이번엔 합칩니다. 팀을 꾸리고, AI 챗봇 서비스를 기획부터 배포까지 직접 진행합니다. 리눅스, 웹, DB, AI API를 하나의 흐름으로 연결하는 경험, 그게 이 프로젝트의 목표입니다.

리눅스 환경 구성, 웹 개발, 데이터베이스 구축, AI API 연동 기술은 현대적인 웹 서비스를 만드는 핵심 요소입니다. 이 기술들은 개별적으로도 중요하지만, 하나의 서비스 안에서 유기적으로 통합될 때 비로소 완전한 가치를 발휘합니다.

이 미션에서는 팀 단위 협업을 통해 사용자 문의에 실시간으로 응답하는 AI 챗봇 서비스를 직접 기획하고 개발합니다. 메신저 플랫폼과 연동되거나 독자적인 웹 인터페이스를 가진 애플리케이션을 완성하며, 사용자 질문을 AI 모델에 전달하고 응답을 받아 출력하는 전체 파이프라인을 구축합니다.

이 과정을 통해 분산적으로 학습한 지식들이 실제 서비스에서 어떻게 상호작용하는지 깊이 있게 이해하고, 팀 단위 소프트웨어 개발 및 배포 실무 역량을 확보하게 됩니다.

2. 최종 결과물

다음 2가지 조건을 만족시키고, 정상 동작하는 웹 애플리케이션 1개를 완성한다.

1. 웹 기반 AI 챗봇 서비스 (FastAPI)

- 입력/요청: 사용자가 웹 페이지에서 텍스트 질문을 입력한다. (로그인 상태)
- 처리: 서버가 질문을 수신하고 AI API를 호출해 응답을 생성한다.
- 출력/화면: AI 응답이 웹 화면에 표시된다.

- 접근성: 평가 시점에 외부 네트워크에서 접속 가능한 서비스 URL이 제공되어야 한다.

2. 프로젝트 산출물 (검증 가능 패키지)

- GitHub Repository 링크
- README/기술 문서
 - 프로젝트 개요(문제 정의, 타겟 사용자, 핵심 시나리오)
 - 시스템 구조(간단 아키텍처, 주요 컴포넌트 역할)
 - API 명세(요청/응답 예시 포함)
 - DB 구조(ERD 또는 테이블/필드 설명)
 - 배포/실행 방법(환경 변수 설정 포함)
 - 팀 구성원 역할 및 개인별 작업 요약
 - 민감정보 관리(.env 예시 제공, .gitignore 적용 등)
- DB 확인 가이드 (가능 범위 내, 아래 중 1개 이상)
 - 로그 조회 API 제공 및 예시 요청
 - 관리자/내부 로그 확인 화면 제공
 - 확인용 SQL 또는 스크립트 제공 (또는 캡처/증빙 자료 링크)

3. 과제 목표

이 과제를 마친 후, 학습자는 아래 내용을 스스로 설명할 수 있어야 한다.

- FastAPI로 웹 서비스(라우팅, 요청/응답, 템플릿 또는 프론트 연동)를 구성하는 흐름을 설명할 수 있다.
- 회원가입/로그인과 인증 상태에 따른 접근 제어를 서비스에 적용한 이유와 방식을 설명할 수 있다.
- “질문 수신 → AI API 호출 → 응답 반환 → 대화 로그 저장” 파이프라인을 설명할 수 있다.
- 대화 로그를 DB에 누적 저장하고 사용자 기준으로 조회/추적 가능하게 만든 설계를 설명할 수 있다.
- AI API 실패/타임아웃 상황에서 서비스가 비정상 종료되지 않도록 처리하고, 로그로 원인을 추적하는 방법을 설명할 수 있다.
- 브랜치 전략, PR 기반 협업, 문서화를 통해 팀 개발 품질을 유지하는 방법을 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 웹 UI (질문 입력 인터페이스)

- 사용자가 질문을 입력할 수 있는 웹 페이지가 존재해야 한다.

- 질문 입력 후 응답이 같은 화면에서 확인 가능해야 한다. (형태 자유: 단일 페이지/페이지 전환 등)

2. 사용자 인증 및 접근 제어

- 회원가입 및 로그인 기능이 정상 동작해야 한다.
- 인증 상태(로그인/비로그인)에 따라 접근 가능한 기능이 구분되어야 한다.
- “챗봇 질문/응답 기능”은 로그인한 사용자만 사용할 수 있어야 한다.

3. AI 챗봇 처리 (FastAPI 백엔드)

- 서버가 사용자 질문을 수신하고, AI API를 호출하여 응답을 생성해야 한다.
- AI API 호출은 서버에서 수행되어야 하며(키 노출 방지), 결과만 화면에 반환되어야 한다.
- 문맥 유지가 가능하도록, 최소한의 컨텍스트 구성 전략을 적용해야 한다.
 - 예: 최근 N개의 대화, 같은 사용자/세션의 직전 Q/A 등 (구현 방식은 팀이 선택)

4. 대화 로그 저장 및 조회/추적

- 사용자 질문과 AI 응답이 DB에 누적 저장되어야 한다.
- 최소 추적 필드: 사용자 식별, 생성 시각, 질문, 응답
- “사용자 기준 로그 조회/추적”이 가능해야 한다.
 - 예: 내 로그 조회 API/화면, 관리자 조회 API/화면, 또는 확인용 쿼리/스크립트 제공(형태 자유)

5. 운영 및 유지보수 (로그/예외/입력 검증)

- 서버 로그에 최소한 아래 이벤트가 남아야 한다.
 - 요청 수신 / AI 호출 / AI 응답 수신(또는 실패) / DB 저장 성공·실패
- AI API 실패/타임아웃 상황에서 서비스가 비정상 종료되지 않아야 한다.
- 사용자에게 오류가 발생했음을 알리는 응답(메시지/상태코드/안내)이 제공되어야 한다.
- 사용자 입력 검증 로직이 최소 1개 이상 존재해야 한다.
 - 예: 빈 입력 차단, 길이 제한 등

6. 배포 및 접근성

- 배포된 서비스는 평가 시점 기준 외부 네트워크에서 접속 가능해야 한다.
- 배포/실행 방법과 환경 변수 설정 방법이 문서에 포함되어야 한다.

7. 협업 및 형상관리

- 브랜치 전략이 적용되어야 한다. (예: main/develop 분리 또는 이에 준하는 운영)
- 기능 단위 작업 브랜치(또는 이에 준하는 작업 흐름) 흔적이 있어야 한다.
- PR 기반 Merge 기록이 존재해야 한다.
- 팀원별 유의미한 커밋이 10회 이상 존재해야 한다.
- 문서에 팀 역할/개인별 작업 요약이 포함되어야 하며, Git 이력과 크게 모순되지 않아야 한다.

개발환경

5. 개발 환경

- Python & FastAPI로 구현한다.
- SQLite 사용을 권장하며, 평가자가 확인 가능한 형태로 연결/조회가 가능해야 한다

제약조건

6. 제약 사항

- 민감정보 및 설정 관리
 - API 키, DB 비밀번호 등 민감정보를 코드/문서에 직접 작성하지 않는다.
 - 모든 민감정보는 환경 변수(.env 등)로 관리한다.
 - .env 파일은 저장소에 업로드되지 않도록 .gitignore를 적용한다.
 - README에 환경 변수 키 목록(이름 수준)과 설정 방법을 포함한다.
- 운영 안정성
 - AI API 호출은 타임아웃을 설정해야 하며(값은 팀이 선택), 실패 시 사용자에게 오류 안내를 반환해야 한다.
 - 요청 수신, AI 호출/응답, DB 저장 성공·실패가 로그로 남아야 한다.
- 협업 규칙
 - PR 기반으로 머지하고, 브랜치/PR 기록이 저장소에 남아야 한다.
 - 팀원별 유의미한 커밋 최소 10회 이상 기록되어야 한다.
 - 기술 문서에 팀 구성원 역할 및 개인별 작업 요약을 반드시 포함해야 한다.

7. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 디자인은 얼마든지 달라도 된다.

- 웹 화면 예시 (대화 흐름)

(로그인 후)

사용자: "내가 방금 뭘 물어봤지?"

챗봇: "직전에 '배포 방법'을 물어보셨고, 저는 다음과 같이 답했습니다: ..."

CSS

- 에러 응답 예시 (타임아웃/실패)

사용자: "긴 글 요약해줘"

챗봇: "현재 응답이 지연되고 있어요. 잠시 후 다시 시도해 주세요. (error: AI_TIMEOUT)"

CSS

- 서버 로그 예시 (핵심 이벤트)

INFO request_received user_id=12 path=/api/chat

INFO ai_call_start user_id=12 request_id=abc123

INFO ai_call_success request_id=abc123 latency_ms=1240

INFO db_save_success user_id=12 chat_id=987

CSS

- 로그 조회 예시(형태 자유)

예시 1) 내 대화 로그 조회 API를 제공하는 경우

- GET /api/me/chats

- 응답에 질문/응답/시간이 포함되어 화면 또는 JSON으로 확인 가능

예시 2) 확인용 SQL/스크립트를 제공하는 경우

- scripts/check_logs.sql 실행 시 최근 대화 로그가 조회되도록 구성

CSS

